

Verilog (week 3): some assembly required

Back to Basics: Module Definition

- Module definition syntax

```
module priority_decoder_4to2 (  
    input wire [3:0] in,          // 4 input request lines  
    output reg [1:0] code,        // encoded index of highest-priority input  
    output reg          valid     // 1 if any input is asserted  
> ); ...  
  
endmodule
```

- `input/output` are keywords to define direction of port (`inout` port can be used for both)
- `// ...` and `/* */` are for comments
- `wire/reg` define data types and `[N:0]` defines size; we will get to them later
- Each module definition must define ports (inputs & outputs) in parentheses
- Module definition ends with `endmodule`
- **Note:** Curly braces are replaced with explicit `begin, end...`-style statements in Verilog

Back to Basics: Instantiation

- define variables for **input/output**
for eg: `req_lines`, `priority_code`,
`req_valid`
- instantiate it with a **unique instance name**
for eg: `pdec_inst` is the unique identifier for this
instance of `priority_decoder_4to2`
- wire in the inputs/outputs using **port names**
for eg: `.in(req_lines)` wires in `req_lines` to the `in` port

```
reg [3:0] req_lines;    // input - reg
wire [1:0] priority_code; // output - wire
wire      req_valid;    // output - wire

priority_decoder_4to2 pdec_inst (
    .in      (req_lines),
    .code    (priority_code),
    .valid    (req_valid)
);
```

Back to Basics: Can you C it?

Most C-style **operators** are supported:

- Bitwise: `& | ^ ~` — per-bit operations
- Logical: `&& || !` — collapse to true/false, used in conditions
- Comparison: `== != < >`
- Concatenation: `{a, b}` — combines signals into a wider signal vector

Control constructs (allowed inside `always` / `initial` only)

- `if / else` — same as software
- `case` — switch-statement for hardware

Back to basics: One isn't good enough

Using vector signals in verilog is easy:

- **Declaring:** `wire/reg [MSB:LSB] var_name`

for eg: `wire [3:0] data` declares a wire that can carry 4 bits

- **Literals:** `<width>'<base><value>`

for eg: `4'b1010` (binary), `8'hA5` (hex), `4'd9` (decimal)

- **Assignment:**

`data = 4'b0110` (full assignment)

`data[1:0] = 2'b10` (multi-bit assignment)

`data[3:0] = {2'b01, 2'b10}` (using concat operator)

> this will assign the bits 3, 2, 1, 0 (left to right) to the values 0, 1, 1, 0.

- **Reading:** Same syntax as assignment

for eg: `data[high:low]` will read bits `low, low+1, ... high-1, high`

- Note: You can use the `[LSB:MSB]` convention as well in Verilog – stay consistent with one convention.

Back to Basics: More data types

Verilog provides helper data types to make life easier – these do not map to real hardware (`wire/reg` do).

- **integer** – plain whole number variable, mostly used to count things (like loop iterations)

for eg: `integer i; for (i = 0; i < 16; i = i + 1) .`

- **parameter** – a named constant you can **reconfigure** from outside the module

for eg: `parameter WIDTH = 8` – can write variable-width modules using this

- **real** – a decimal/floating-point number; **only used in simulation/testing**

for eg: `real voltage = 3.3 .`

- **time** – stores a **simulation/testing timestamp**, usually paired with `$time`

for eg: `time start_time; start_time = $time; .`

- **localparam** – a named constant that is **fixed inside a module** – used to store helper values

for eg: `localparam OP_ADD = 4'b0000; .`

Theory Terms

- Recall the following from CS230:
 - **Clock:** A periodic signal (typically square wave) that triggers updates in sequential circuits.
 - **Level-triggered:** A circuit that responds continuously **while a control signal is high** (or low).
 - **Edge-triggered:** A circuit that responds only at **an instant** — when the clock transitions from **0→1 (posedge)** or **1→0 (negedge)**
- **D Flip-Flop (edge-triggered):** A circuit that samples its input on a clock edge and holds that value until the next edge
- **T Flip-Flop (edge-triggered):** A circuit that toggles its output (flips $0 \leftrightarrow 1$) when both the clock edge arrives and the toggle input is 1

Back to basics: Data Types

Verilog supports two data types for these purposes:

- **wire** – carries a value **continuously** from whatever drives it; has no memory of its own
 - **assign** is used for **continuous assignment** to a wire. It cannot be used for register data.
 - Eg: `assign q_bar = ~q_reg;`
- **reg** – a variable that **holds its value** until explicitly assigned inside a procedural block
 - **Note:** The definition is **logical**. It is not guaranteed to synthesize to a **physical register**
 - Can only be assigned inside of `always @(...)` blocks
 - Usage: `always @(trigger-list)`
 - Eg: `always @(posedge clk)` executes on **rising edge of clk** signal -> **sequential**
 - Eg: `always @(*)` executes whenever **any input changes** -> **combinational**

Coming back to you (always)

Combinational

- requires **no physical registers**
- **ALL updates are executed at once**
- (typically) blocking assignment (=) is used
- written inside **always @(*)** blocks
- the blocking statements run in **program order**
- Eg: some random combinational logic here

```
always @(*) begin
    c = a & b;
    d = a | b;
    out1 = d;
    out2 = c ^ b;
end
```

Sequential

- (usually) requires **physical registers**
- updates transferred **sequentially ONE-by-ONE**
- (typically) non-blocking assignment (<=) is used
- written inside **always @(posedge clk)** blocks (or a variant)
- Eg: naive 2-bit (modulo 4) counter

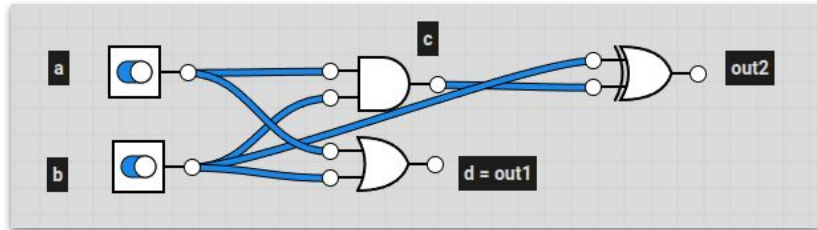
```
always @(posedge clk) begin
    if (reset)
        count <= 2'b00;
    else
        count <= count + 2'b01;
end
```

Coming back to you (always)

Combinational

```
always @(*) begin
    c = a & b;
    d = a | b;
    out1 = d;
    out2 = c ^ b;
end
```

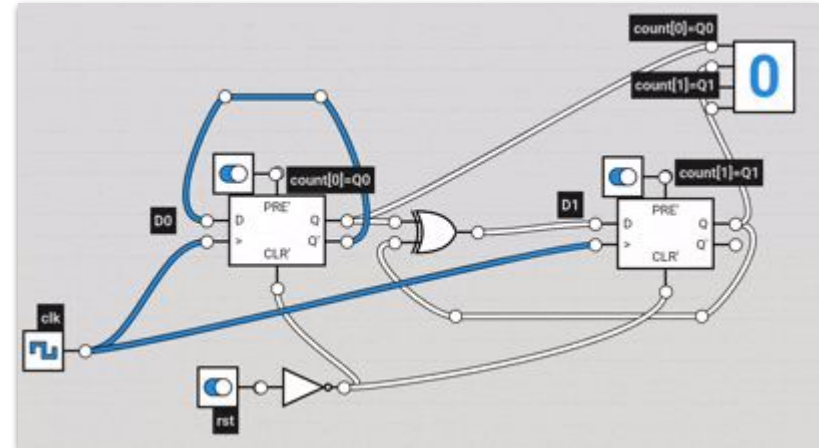
- Circuit will respect the program order for computation



Sequential

```
always @(posedge clk) begin
    if (reset)
        count <= 2'b00;
    else
        count <= count + 2'b01;
end
```

- Circuit will respect the clock-based update



How NOT to TFF it!

1. tff_wire

Need **reg** not **wire**; Need **edge-triggered** not **level-triggered**

```
module tff_wire (  
    input clk,  
    input t,  
    output wire q  
);  
    assign q = clk ? (t ^ q) : q;  
endmodule
```

2. tff_reg

Blocking assignment gives wrong assignment, need **<=**

```
module tff_reg (  
    input clk,  
    input t,  
    output reg q  
);  
    always @(posedge clk) begin  
        q = t ^ q;  
    end  
endmodule
```

Why you should not block (me)

- `<=` stands for non-blocking assignment – this allows us to defer updates to the end and execute them together
- How it works at a high level:
 - Read all old values (occurring on right hand side first)
 - Then update all left hand side values **together**
- Why is it called “non-blocking”?

All writes are **scheduled** to happen later, i.e., it **doesn't block** any other statement trying to read the **old value**.

A blocking assignment (`=`) will write the new value immediately and block anyone else trying to read the old value.
- **Rule of thumb:** Always use `<=` inside `always @(posedge clk)` or some variant.
- **Note:** `<=` and `=` (other than `assign <wire> = statements`) can only be used in `always` or `initial` blocks.

Example: DFF

```
module dff_with_inverter (  
    input wire clk,  
    input wire rst_n,  
    input wire d,  
    output wire q,  
    output wire q_bar  
);  
  
    reg q_reg;  
  
    // ---- Sequential logic ----  
    // Output depends on current input AND past state (memory)  
    // Only updates on the clock's rising edge  
    always @(posedge clk) begin  
        if (!rst_n)  
            q_reg <= 1'b0;  
        else  
            q_reg <= d;  
        end  
  
    // ---- Combinational logic ----  
    // Output depends only on current input, no memory, no clock  
    assign q = q_reg;  
    assign q_bar = ~q_reg;  
  
endmodule
```

Example: DFF testbench

```
`timescale 1ns/1ns

module tb_dff_with_inverter;

    reg clk, rst_n, d;
    wire q, q_bar;

    dff_with_inverter dut (
        .clk(clk), .rst_n(rst_n),
        .d(d), .q(q), .q_bar(q_bar)
    );

    // Clock generation: 10ns period
    always #5 clk = ~clk;

    initial begin
        $dumpfile("dff_wave.vcd");
        $dumpvars(0, tb_dff_with_inverter);

        clk = 0; rst_n = 0; d = 0;

        #12 rst_n = 1; // release reset
    end
endmodule
```

```
        initial begin
            $dumpfile("dff_wave.vcd");
            $dumpvars(0, tb_dff_with_inverter);

            clk = 0; rst_n = 0; d = 0;

            #12 rst_n = 1; // release reset

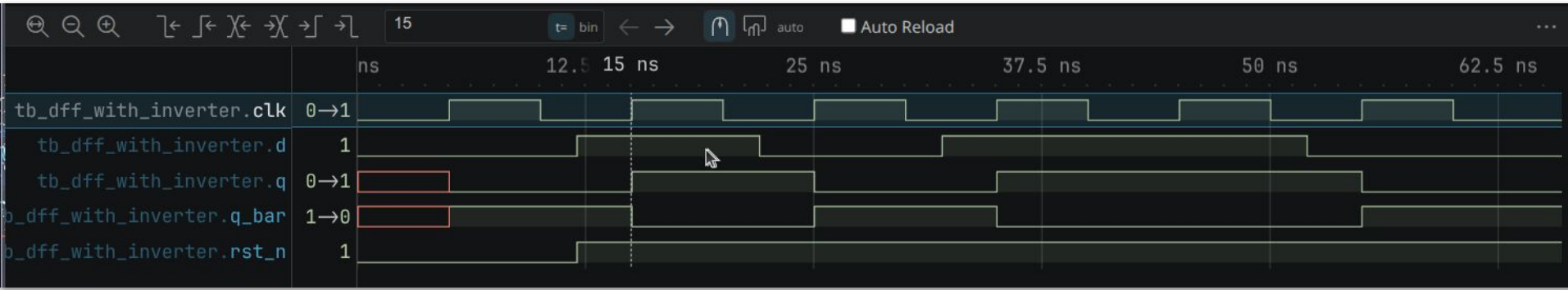
            d = 1; #10;
            d = 0; #10;
            d = 1; #10;
            d = 1; #10;
            d = 0; #10;

            $display("Test complete.");
            $finish;
        end

        initial begin
            $monitor("t=%0t rst_n=%b d=%b q=%b q_bar=%b",
                $time, rst_n, d, q, q_bar);
        end
    end

endmodule
```

Example: DFF Waveform



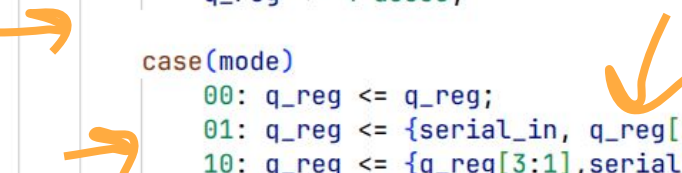
- An example testbench waveform for the D-flip flop implementation
- Note clock time period is 10ns
- `d` update is propagated to `q` only on **rising edge** when `rst_n` is set

These should be doubts

Common problems you guys ran into in the last labexam:

1. A **if** and **case** always occur inside an **always** block
2. Binary literals need to be preceded with **2'b** here
3. Do not miss the **else** clause here
4. Be careful with bit indexing for left and right shift
5. Don't run **initial q = 4'b0** in the module, let **rst** handle it

```
module universal_shift_reg (  
    input clk,  
    input rst,  
    input [1:0] mode,  
    input serial_in,  
    input [3:0] parallel_in,  
    output [3:0] q  
);  
  
    // TODO: Implement the universal shift register  
    reg [3:0] q_reg;  
    assign q = q_reg;  
  
    always @(posedge clk) begin  
        if(rst)  
            q_reg <= 4'd0000;  
  
        case(mode)  
            00: q_reg <= q_reg;  
            01: q_reg <= {serial_in, q_reg[2:0]};  
            10: q_reg <= {q_reg[3:1], serial_in};  
            11: q_reg <= parallel_in;  
            default: q_reg <= q_reg;  
        endcase  
    end  
endmodule
```



PS: Resetting Your Counts to Zero

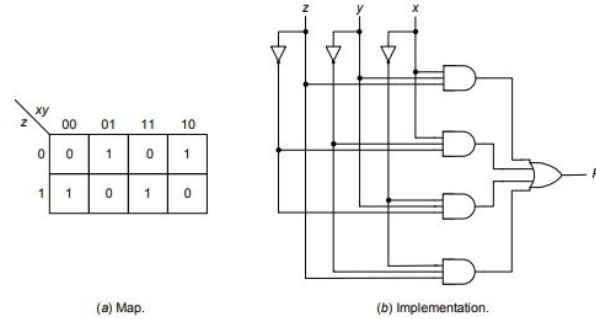
- **Reset:** A signal used to reset the register values to zero
- **Synchronous Reset:** Only takes effect when clock signal permits (for eg on posedge clk)
 - When we used `always @(posedge clk)` and `if(rst)`, we were using this pattern
- **Asynchronous Reset:** Takes effect whenever updated – independent of clock signal
 - More common in practice. Used as `always @(posedge clk or posedge rst)`
- The above is called an “**active-high**” reset which resets when the `rst` signal goes high
- In practice, “**active-low**” reset is more common, i.e., it resets when the `rst_n` signal goes low
- Use it as `always @(posedge clk or negedge rst_n)` -- we will use this more frequently

PS: Know your geography

- Recall Karnaugh maps
- Why do we use them?
- What are don't-cares?
- Can you simplify boolean functions?

Combinational Circuits: Parity-bit Generator

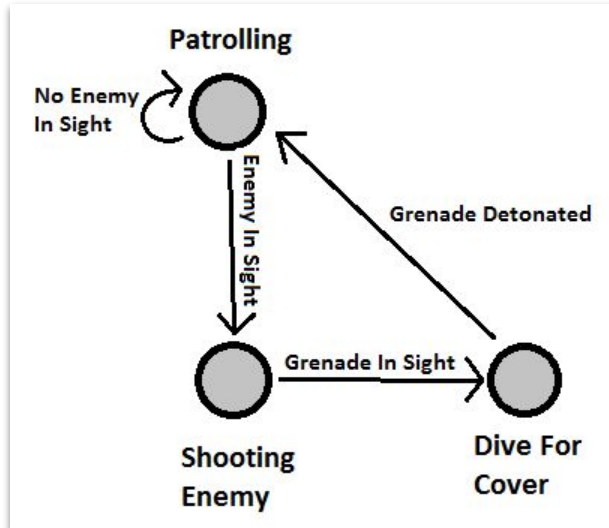
Parity-bit generator: produces output value 1 if and only if an odd number of its inputs have value 1



$$P = x'y'z + x'yz' + xy'z' + xyz = x \oplus y \oplus z$$

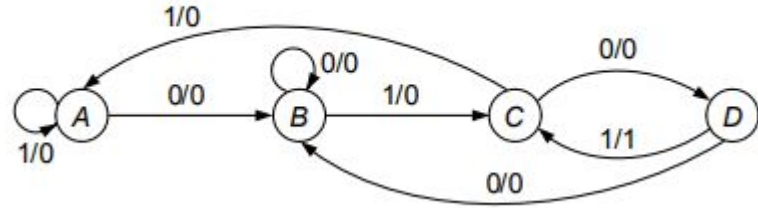
Why more Verilog?

- We will cover hardware design from a high level today!
- Also a common idiom in hardware design: finite state machines.
- **Finite state machines** define well-defined states (memory) and transitions (combinational logic) between them to consume some input and produce some output.
- Example: Bot AI in games be like



Eg: Simple 0101 detector

State diagram and state table:



Plan for this lab

- Designing and writing a simple FSM-based circuit together
- First discuss, later you attempt as task0
- **Goals:**
 - Convert specifications to **high-level component design**
 - Write an **FSM** in Verilog: states, transitions, outputs, counters
 - **Combine smaller modules** to solve the entire problem



Traffic Control Simulator

P1. Bhavesh wants to go to his favourite restaurant right across JVLR road. But the traffic is horrible. Bhavesh is annoyed and blames the traffic light controller. He wishes to design a better traffic control system!

Goal: Build a chip that cycles through **red**->**green**->**yellow** with some fixed duration for each, and takes a pedestrian button as input – when pressed by a pedestrian, reduce the duration of **green** and increase that of **red**.

- **Q1:** What external inputs do we need? tick tock...
- **Q2:** What outputs do we need to produce?
- **Q3:** Do we need to store state for the circuit?
- **Q4:** What transitions are possible?
- **Q5:** How do we decide when we need to switch states?

Traffic Control Simulator

- **Q1:** What external inputs do we need? tick tock...
 - Sequential/state-based circuit -> We need **clk**, **rst_n**
 - Button for pedestrian -> **ped_button**
 - Is clk a good way to keep track of time? Why/why not?
 - We need **tick_1hz**
- **Q2:** What outputs do we need to produce?
 - One wire for each output: **red**, **yellow**, **green**

```
module bhavesh_traffic_light (  
    input clk, rst_n,  
    input ped_button,  
    input tick_1hz,  
  
    output red, green, yellow,  
);
```

Abstraction!

- We have settled on input, output and some internal counters/tracking variables
- We need to model:
 - Updating **state** based on time passed vs duration of a light
 - Updating **elapsed time** based on tick signal
 - Updating **duration** of a light based on pedestrian button
- Assume **8-bit durations**:
 - RED: Normally, **30** / If pedestrian button pressed, **40**
 - GREEN: Normally, **25** / If pedestrian button pressed, **15**
 - YELLOW: Always **5**
- Draw states, transitions and input/outputs